Algoritmi e Strutture Dati – Pile e Code

Summary operativo su Stack, Queue e implementazione tramite Liste Monodirezionali

Riferimento: Demetrescu, Finocchi, Italiano. Algoritmi e Strutture Dati – Seconda Edizione

1 Introduzione

Le pile (stack) e le code (queue) sono strutture dati dinamiche lineari che rappresentano casi particolari delle liste collegate (in particolare, delle liste monodirezionali o *singly linked lists*).

A differenza di una lista generica, in cui gli elementi possono essere inseriti, cercati o rimossi in qualsiasi posizione, pile e code impongono rigide politiche di accesso ai dati:

- **Pila (Stack):** Politica **LIFO** (*Last-In, First-Out*). L'ultimo elemento inserito è il primo a essere rimosso.
- **Coda (Queue):** Politica **FIFO** (*First-In, First-Out*). Il primo elemento inserito è il primo a essere rimosso.

Entrambe possono essere implementate in modo efficiente sfruttando i nodi e i puntatori tipici delle liste collegate, garantendo operazioni in tempo costante $O(1)$ nel caso peggiore (senza i problemi di ridimensionamento tipici degli array).

2 La Pila (Stack)

Una pila gestisce i dati consentendo inserimenti e rimozioni da un'unica estremità, comunemente chiamata **cima** o **top**.

2.1 Implementazione con Liste Monodirezionali

Utilizzando una lista monodirezionale, la cima della pila coincide con la testa (`head`) della lista:

- L'**inserimento** (`push`) corrisponde a un inserimento in testa ($O(1)$).
- La **rimozione** (`pop`) corrisponde a una cancellazione in testa ($O(1)$).

2.2 Operazioni Fondamentali

1. `isEmpty()`**:** Restituisce un valore booleano (`true` o `false`) che indica se la pila è vuota (ovvero se `head` == `null`).
   - **Complessità:** $O(1)$.
2. `push(elem)`**:** Inserisce un nuovo elemento `elem` in cima alla pila.
   - **Descrizione:** Si crea un nuovo nodo contenente `elem`, si imposta il suo puntatore `next` sulla vecchia testa della lista, e si aggiorna `head` affinché punti al nuovo nodo.
   - **Complessità:** $O(1)$.
3. `pop()` $\rightarrow$ **elem:** Rimuove e restituisce l'elemento attualmente in cima alla pila.

- **Descrizione:** Si preleva il valore dal nodo in testa (`head`), quindi si sposta `head` al nodo successivo (`head` ← `head.next`) e si dealloca il vecchio nodo. Se la pila è vuota, si genera una condizione di errore.
- **Complessità:** $O(1)$.

4. `top()` → **elem:** Restituisce l'elemento in cima alla pila senza rimuoverlo.
   - **Descrizione:** Si legge direttamente il valore del nodo puntato da `head`.
   - **Complessità:** $O(1)$.

3 La Coda (Queue)

Una coda gestisce i dati consentendo l'inserimento da un'estremità detta **coda** (*tail*) e la rimozione dall'estremità opposta detta **testa** (*head*).

3.1 Implementazione con Liste Monodirezionali

Per implementare una coda in modo efficiente con una lista monodirezionale, la struttura deve mantenere due puntatori espliciti:

- `head`: punta al primo elemento della coda (da cui si rimuovono gli elementi con `dequeue`).
- `tail`: punta all'ultimo elemento della coda (in cui si inseriscono i nuovi elementi con `enqueue`).

Senza il puntatore `tail`, l'inserimento in coda richiederebbe ogni volta di scorrere tutta la lista ($O(n)$). Con il puntatore `tail`, tutte le operazioni principali costano $O(1)$.

3.2 Operazioni Fondamentali

1. `isEmpty()`: Verifica se la coda è vuota controllando se `head` == `null` (o equivalentemente `tail` == `null`).
   - **Complessità:** $O(1)$.
2. `enqueue(elem)`: Inserisce un nuovo elemento `elem` in fondo alla coda.
   - **Descrizione:** Si crea un nuovo nodo, si aggancia in coda dopo l'attuale `tail`, e si aggiorna `tail` affinché punti al nuovo nodo. Se la coda era vuota, sia `head` che `tail` confluiranno sul nuovo nodo.
   - **Complessità:** $O(1)$.
3. `dequeue()` → **elem:** Rimuove e restituisce l'elemento in testa alla coda.
   - **Descrizione:** Si preleva il valore del nodo in `head`, si sposta `head` al nodo successivo (`head` ← `head.next`). Se dopo la rimozione la coda diventa vuota, si aggiorna anche `tail` ← `null`. Se la coda era già vuota, si genera un errore.
   - **Complessità:** $O(1)$.
4. `first()` → **elem:** Restituisce l'elemento in testa alla coda senza rimuoverlo.
   - **Descrizione:** Si legge il valore del nodo puntato da `head`.
   - **Complessità:** $O(1)$.

4 Riepilogo delle Complessità Temporali

Operazione	**Pila (Stack)**	**Coda (Queue)**
`isEmpty()`	$O(1)$	$O(1)$
Inserimento (`push` / `enqueue`)	$O(1)$	$O(1)$*
Rimozione (`pop` / `dequeue`)	$O(1)$	$O(1)$
Lettura elemento di accesso (`top` / `first`)	$O(1)$	$O(1)$

**Nota: Per la coda, l'inserimento in tempo $O(1)$ richiede l'uso del puntatore esplicito `tail`.*